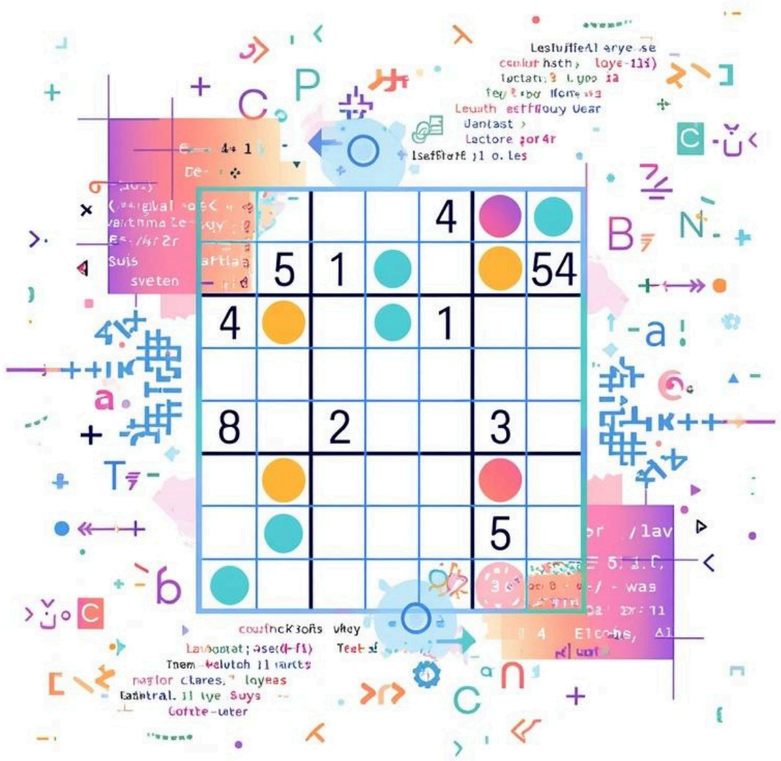
C Mini Project : Multi grid Sudoku Game

(4x4 and 9x9)



PRESENTED BY : ARYA CHALIKWAR : 1262251699
DEETYA PANCHAL : 1262251777
HELLY GANDHI : 1262250532
SHAMBHAVI KATYAYAN : 1262252522

Team Roles Overview

LOGIC DESIGN

Shambhavi was responsible for creating the overall structure, focusing on **developing algorithms** and ensuring adherence to Sudoku rules.

CODING IMPLEMENTATION

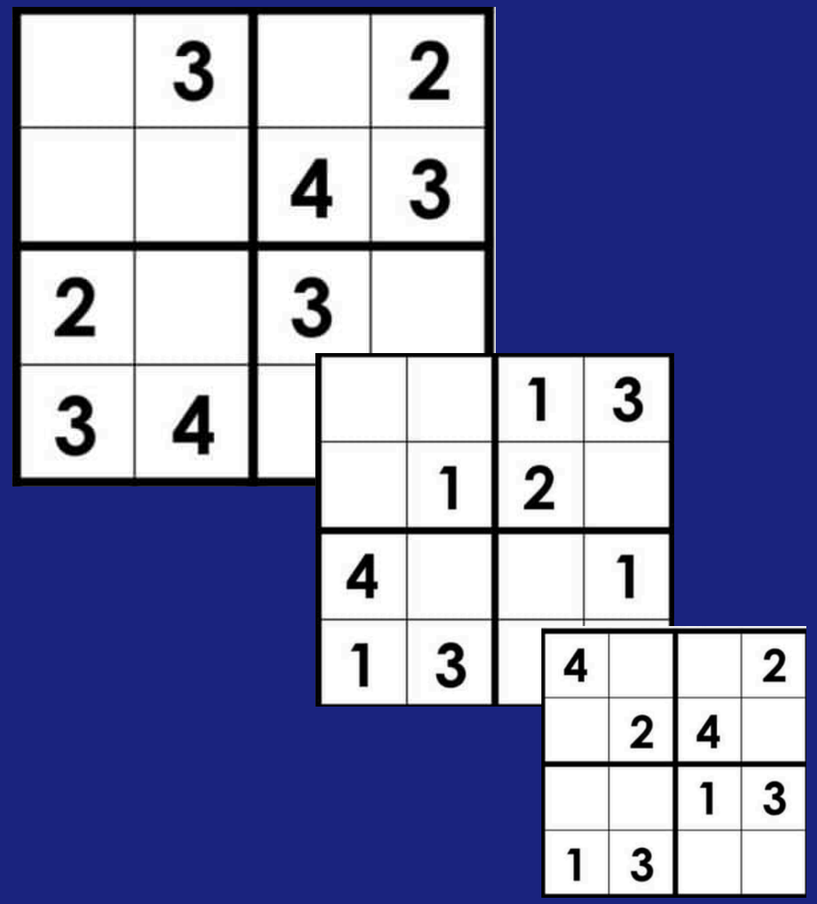
Helly and Arya took charge of writing the core C program, expertly implementing features **to streamline user interaction** and maintain efficiency.

TESTING PROCESS

Deetya conducted thorough testing, identifying bugs and enhancing performance, ensuring the final product was **reliable and user-friendly** for all players.

Key Features of Our Sudoku Game

- INTERACTIVE 4×4 AND 9×9 SUDOKU GRID FOR USERS.
- AUTOMATICALLY GENERATES A NEW SOLVABLE SUDOKU PUZZLE EVERY TIME.
- DIFFERENT PUZZLE EACH TIME WHEN USER ENTERS -1.
- DETECTS WRONG NUMBERS AND SHOWS WARNING MESSAGES.
- DISPLAYS A CONGRATULATORY MESSAGE WHEN SUDOKU IS COMPLETED.
- ENSURES SUDOKU RULES (NO REPETITION IN ROWS AND COLUMNS).
- USER-FRIENDLY INPUT SYSTEM .
- REPLAY OPTION WITHOUT RESTARTING THE PROGRAM.
- ENTER -2 TO DISPLAY THE COMPLETE SOLUTION OF THE CURRENT SUDOKU PUZZLE.
- IMPLEMENTED USING CORE C PROGRAMMING CONCEPTS LIKE ARRAYS, LOOPS, CONDITIONS, AND FUNCTIONS.



Understanding How Our Sudoku Game Functions

GAME FLOW OVERVIEW

1. The program starts and generates a random solvable 4×4 and 9×9 Sudoku grid.
2. The Sudoku board is displayed with some cells empty.
3. The user enters the row, column, and number to fill a cell.
4. The program checks whether the entered number follows Sudoku rules.
5. If valid, the number is placed in the grid.
6. If invalid, a “Wrong number” message is displayed.
7. The process continues until all empty cells are filled.
8. When the Sudoku is completed, a congratulatory message is shown.
9. The user can enter -1 to generate a new Sudoku puzzle and play again, -2 to check the solution and 0 to exit the game.

C-CODE

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

#define MAX 16

int N;
int board[MAX][MAX];
int solution[MAX][MAX];
int SRN;

// Print board
void printBoard() {
    printf("\nBoard:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i][j] == 0)
                printf(". ");
            else
                printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

// Check validity
int isValid(int row, int col, int num) {
    for (int x = 0; x < N; x++) {
        if (board[row][x] == num) return 0;
        if (board[x][col] == num) return 0;
    }

    int startRow = row - row % SRN;
    int startCol = col - col % SRN;

    for (int i = 0; i < SRN; i++)
        for (int j = 0; j < SRN; j++)
            if (board[startRow + i][startCol + j] == num)
                return 0;

    return 1;
}

// Shuffle numbers
void shuffle(int *arr) {
    for (int i = 0; i < N; i++) {
        int j = rand() % N;
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

// Backtracking generator
int fillGrid(int row, int col) {
    if (row == N)
        return 1;

    if (col == N)
```

```c
        return fillGrid(row + 1, 0);

    int nums[MAX];
    for (int i = 0; i < N; i++)
        nums[i] = i + 1;

    shuffle(nums);

    for (int i = 0; i < N; i++) {
        int num = nums[i];

        if (isValid(row, col, num)) {
            board[row][col] = num;

            if (fillGrid(row, col + 1))
                return 1;

            board[row][col] = 0;
        }
    }

    return 0;
}

// Remove digits
void removeDigits(int count) {
    while (count > 0) {
        int r = rand() % N;
        int c = rand() % N;
```

```c
        if (board[r][c] != 0) {
            board[r][c] = 0;
            count--;
        }
    }
}

// Generate Sudoku
void generateSudoku() {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = 0;

    fillGrid(0, 0);

    // Save solution
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            solution[i][j] = board[i][j];

    removeDigits(N * N / 2);
}

// Check completion
int isComplete() {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            if (board[i][j] == 0)
                return 0;
    return 1;
```

```c
int main() {
    int r, c, n;

    srand(time(0));

    printf("Enter size (4 or 9): ");
    scanf("%d", &N);

    SRN = sqrt(N);

    if (SRN * SRN != N) {
        printf("Invalid size! Must be perfect square.\n");
        return 0;
    }

    generateSudoku();

    while (1) {
        printBoard();

        // 🎉 Completion check
        if (isComplete()) {
            printf("\n🎉 Congratulations! You solved the Sudoku!\n"
                );
            printf("Enter -1 for new game or 0 to exit.\n");
        }

        printf("\nEnter row col num (1-%d) or -1 for new or -2
            solution or 0 exit: ", N);
```

```c
// 🚪 Exit
if (r == 0) {
    printf("\n👋 Thanks for playing Sudoku!\n");
    break;
}

// 🔁 New game
if (r == -1) {
    generateSudoku();
    continue;
}

// 🧠 Show solution
if (r == -2) {
    printf("\nSolution:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            printf("%d ", solution[i][j]);
        printf("\n");
    }
    continue;
}
```

```c
    scanf("%d %d", &c, &n);

    r--;
    c--;

    if (r >= 0 && r < N && c >= 0 && c < N &&
        board[r][c] == 0 && n >= 1 && n <= N) {

        if (isValid(r, c, n))
            board[r][c] = n;
        else
            printf("❌ Invalid move! Violates Sudoku rules.\n");

    } else {
        printf("❌ Wrong input!\n");
    }
}

return 0;
```

Program Output Examples and Messages

Output

```
Enter size (4 or 9): 9

Board:
1 . . . 4 . . . .
9 5 . . . . . 2 4
. . 7 2 . 9 1 5 3
. . 2 4 8 . 9 1 5
. . . 5 . 1 3 6 2
5 1 . . . . 8 . 7
7 4 . 1 5 3 . 9 6
. . . . 7 4 5 . 8
3 . . 8 . . 4 . .

Enter row col num (1-9) or -1 for new or -2 solution or 0 exit:
```

```
Enter row col num (1-4) or -1 for new or -2 solution or 0 exit: 1 3 1

Board:
2 3 1 .
4 1 3 2
1 2 4 3
3 4 2 .

Enter row col num (1-4) or -1 for new or -2 solution or 0 exit: 1 4 4

Board:
2 3 1 4
4 1 3 2
1 2 4 3
3 4 2 .

Enter row col num (1-4) or -1 for new or -2 solution or 0 exit: 4 4 1

Board:
2 3 1 4
4 1 3 2
1 2 4 3
3 4 2 1

🎉 Congratulations! You solved the Sudoku!
Enter -1 for new game or 0 to exit.
```

What We Learned...

FROM LOGIC TO CODE, AND FROM CODE TO SOLUTION THIS PROJECT SHAPED OUR THINKING.

- Used 2D arrays for Sudoku
- Applied backtracking algorithm
- Checked row, column, grid rules
- Used random puzzle generation
- Created modular functions
- Handled user input and errors
- Built interactive console game

Thank You!